

Sorting Retrieved Data

In this chapter, you will learn how to use the `SELECT` statement's `ORDER BY` clause to sort retrieved data as needed.

Sorting Data

As you learned in the previous chapter, the following SQL statement returns a single column from a database table. But look at the output. The data appears to be displayed in no particular order.

► Input

```
SELECT prod_name  
FROM products;
```

► Output

```
+-----+  
| prod_name |  
+-----+  
| .5 ton anvil |  
| 1 ton anvil |  
| 2 ton anvil |  
| Oil can |  
| Fuses |  
| Sling |  
| TNT (1 stick) |  
| TNT (5 sticks) |  
| Bird seed |  
| Carrots |  
| Safe |  
| Detonator |  
| JetPack 1000 |  
| JetPack 2000 |  
+-----+
```

Actually, the retrieved data is not displayed in a merely random order. Unsorted data is typically displayed in the order in which it appears in the underlying tables. This could

be the order in which the data was added to the tables initially. However, if data was subsequently updated or deleted, the order is affected by how MySQL reuses reclaimed storage space. The end result is that you cannot (and should not) rely on the sort order if you do not explicitly control it. Relational database design theory states that the sequence of retrieved data cannot be assumed to have significance if ordering was not explicitly specified.

New Term

Clauses These are parts of SQL statements are made up of clauses, some required and some optional. A clause usually consists of a keyword and supplied data. An example of this is the `SELECT` statement's `FROM` clause, which you saw in the previous chapter.

To explicitly sort data retrieved using a `SELECT` statement, you use the `ORDER BY` clause. `ORDER BY` takes the name of one or more columns by which to sort the output. Look at the following example:

► Input

```
SELECT prod_name
FROM products
ORDER BY prod_name;
```

► Analysis

This statement is identical to the earlier statement, except that it also specifies an `ORDER BY` clause instructing MySQL to sort the data alphabetically based on the `prod_name` column. The results are as follows:

► Output

```
+-----+
| prod_name |
+-----+
| .5 ton anvil |
| 1 ton anvil |
| 2 ton anvil |
| Bird seed |
| Carrots |
| Detonator |
| Fuses |
| JetPack 1000 |
| JetPack 2000 |
| Oil can |
| Safe |
| Sling |
| TNT (1 stick) |
| TNT (5 sticks) |
+-----+
```

Tip

Sorting by Nonselected Columns More often than not, the columns used in an `ORDER BY` clause are ones that were selected for display. However, this is actually not required, and it is perfectly legal to sort data based on a column that is not retrieved.

Sorting by Multiple Columns

It is often necessary to sort data by more than one column. For example, if you are displaying an employee list, you might want to display it sorted by last name and first name (that is, first sort by last name and then, within each last name, sort by first name). This is useful, for example, if there are multiple employees with the same last name.

To sort by multiple columns, simply specify the column names separated by commas, just as you do when you are selecting multiple columns.

The following code retrieves three columns and sorts the results by two of them—first by price and then by name:

► Input

```
SELECT prod_id, prod_price, prod_name
FROM products
ORDER BY prod_price, prod_name;
```

► Output

prod_id	prod_price	prod_name
FC	2.50	Carrots
TNT1	2.50	TNT (1 stick)
FU1	3.42	Fuses
SLING	4.49	Sling
ANV01	5.99	.5 ton anvil
OL1	8.99	Oil can
ANV02	9.99	1 ton anvil
FB	10.00	Bird seed
TNT2	10.00	TNT (5 sticks)
DTNTR	13.00	Detonator
ANV03	14.99	2 ton anvil
JP1000	35.00	JetPack 1000
SAFE	50.00	Safe
JP2000	55.00	JetPack 2000

It is important to understand that when you are sorting by multiple columns, the sort sequence is exactly as specified. In other words, using the output in this example, the products are sorted based on the `prod_name` column only when multiple rows have the same `prod_price` value. If all the values in the `prod_price` column are unique, no data is sorted by `prod_name`.

Sorting by Column Position

In addition to being able to specify sort order by using column names, you can also use `ORDER BY` to order based on relative column position. The best way to understand this is to look at an example:

► Input

```
SELECT prod_id, prod_price, prod_name
FROM Products
ORDER BY 2, 3;
```

► Output

prod_id	prod_price	prod_name
BNBG02	3.4900	Bird bean bag toy
BNBG01	3.4900	Fish bean bag toy
BNBG03	3.4900	Rabbit bean bag toy
RGAN01	4.9900	Raggedy Ann
BR01	5.9900	8 inch teddy bear
BR02	8.9900	12 inch teddy bear
RYL01	9.4900	King doll
RYL02	9.4900	Queen doll
BR03	11.9900	18 inch teddy bear

► Analysis

As you can see, the output here is identical to that of the query above. The difference here is in the `ORDER BY` clause. Instead of specifying column names, you specify the relative positions of selected columns in the `SELECT` list. `ORDER BY 2` means sort by the second column in the `SELECT` list, the `prod_price` column. `ORDER BY 2, 3` means sort by `prod_price` and then by `prod_name`.

The primary advantage of this technique is that it doesn't require you to retype the column names. But there are some downsides, too. First, not explicitly listing column names increases the likelihood of mistakenly specifying the wrong column. Second, it is easy to mistakenly reorder data when making changes to the `SELECT` list (such as when you forget to make the corresponding changes to the `ORDER BY` clause). And finally, obviously you cannot use this technique when sorting by columns that are not in the `SELECT` list.

Tip

Sorting by Nonselected Columns You cannot use `ORDER BY` to order based on relative column position when sorting by columns that do not appear in the `SELECT` list. However, you can mix and match column names and relative column positions in a single statement, if needed.

Specifying Sort Direction

Data sorting is not limited to ascending sort orders (that is, from A to Z). Although this is the default sort order, you can also use the `ORDER BY` clause to sort in descending order (that is, from Z to A). To sort in descending order, you need to specify the `DESC` keyword.

The following example sorts products by price, in descending order (that is, most expensive first):

► Input

```
SELECT prod_id, prod_price, prod_name
FROM products
ORDER BY prod_price DESC;
```

► Output

prod_id	prod_price	prod_name
JP2000	55.00	JetPack 2000
SAFE	50.00	Safe
JP1000	35.00	JetPack 1000
ANV03	14.99	2 ton anvil
DTNTR	13.00	Detonator
TNT2	10.00	TNT (5 sticks)
FB	10.00	Bird seed
ANV02	9.99	1 ton anvil
OL1	8.99	Oil can
ANV01	5.99	.5 ton anvil
SLING	4.49	Sling
FU1	3.42	Fuses
FC	2.50	Carrots
TNT1	2.50	TNT (1 stick)

But what if you were to sort by multiple columns? The following example sorts the products in descending order (that is, most expensive first) and based on product name:

► Input

```
SELECT prod_id, prod_price, prod_name
FROM products
ORDER BY prod_price DESC, prod_name;
```

► Output

prod_id	prod_price	prod_name
JP2000	55.00	JetPack 2000
SAFE	50.00	Safe
JP1000	35.00	JetPack 1000

ANV03		14.99	2 ton anvil	
DTNTR		13.00	Detonator	
FB		10.00	Bird seed	
TNT2		10.00	TNT (5 sticks)	
ANV02		9.99	1 ton anvil	
OL1		8.99	Oil can	
ANV01		5.99	.5 ton anvil	
SLING		4.49	Sling	
FU1		3.42	Fuses	
FC		2.50	Carrots	
TNT1		2.50	TNT (1 stick)	
+-----+-----+-----+-----+				

► Analysis

The DESC keyword only applies to the column name that directly precedes it. In this example, DESC is specified for the `prod_price` column but not for the `prod_name` column. Therefore, the `prod_price` column is sorted in descending order, but the `prod_name` column (within each price) is still sorted in standard ascending order.

Tip

Sorting Descending on Multiple Columns If you want to sort descending on multiple columns, be sure each column has its own DESC keyword.

The opposite of DESC is ASC (for *ascending*), which you can specify to sort in ascending order. In practice, however, ASC is not usually used because ascending order is the default sequence (and is assumed if neither ASC nor DESC is specified).

Tip

Case-Sensitivity and Sort Orders When you are sorting textual data, is A the same as a? And does a come before B or after Z? These are not theoretical questions, and the answers depend on how the database is set up.

In *dictionary* sort order, A is treated the same as a, and that is the default behavior in MySQL (and, indeed, in most other DBMSs as well). However, administrators can change this behavior, if needed. (If your database contains lots of foreign language characters, for example, this might become necessary.)

The key here is that if you need an alternate sort order, you cannot accomplish it with a simple ORDER BY clause. You must contact your database administrator.

By using a combination of `ORDER BY` and `LIMIT`, it is possible to find the highest or lowest value in a column. The following example demonstrates how to find the value of the most expensive item:

► Input

```
SELECT prod_price
FROM products
ORDER BY prod_price DESC
LIMIT 1;
```

► Output

```
+-----+
| prod_price |
+-----+
|      55.00 |
+-----+
```

► Analysis

`prod_price DESC` ensures that rows are retrieved from most to least expensive, and `LIMIT 1` tells MySQL to return just one row.

Caution

Position of `ORDER BY` Clause When specifying an `ORDER BY` clause, be sure that it is after the `FROM` clause. If `LIMIT` is used, it must come *after* `ORDER BY`. If you use clauses out of order, you will get an error message.

Tip

A Better Way to Find the Largest Value Here we used `ORDER BY` and `LIMIT` to find the most expensive product in a table. In Chapter 12, “Summarizing Data,” we’ll look at a far more efficient way of finding largest (and smallest and average and other) values.

Summary

In this chapter, you learned how to sort retrieved data by using the `SELECT` statement’s `ORDER BY` clause. This clause, which must be the last in the `SELECT` statement, can be used to sort data on one or more columns, as needed.

Challenges

1. Write a SQL statement to retrieve all customer names (`cust_names`) from the `Customers` table and display the results sorted from Z to A.
2. Write a SQL statement to retrieve customer ID (`cust_id`) and order number (`order_num`) from the `Orders` table and sort the results first by customer ID and then by order date, in reverse chronological order.
3. Our fictitious store obviously prefers to sell more expensive items—and lots of them. Write a SQL statement to display the quantity and price (`item_price`) from the `OrderItems` table, sorted with the highest quantity and highest price first.
4. What is wrong with the following SQL statement? (Try to figure it out without running it.)

```
SELECT vend_name,  
FROM Vendors  
ORDER vend_name DESC;
```